EL-2012 Computer Architecture Lab

LABORATORY MANUAL

Fall 2026



Open Ended Lab

Student Name: _____ Roll No: _____ Section: ___

Date performed: _____, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Objective:

After this lab, you will be able to:

- Explain what the Data Memory does inside a RISC-V RV32I processor.
- Build a memory array in Verilog and write data to it using a clock.
- Write a testbench that checks your design on its own.

Introduction

A computer needs two kinds of memory. One keeps the program (the instructions). The other keeps the data that the program uses while it runs. In a RISC-V processor, the second one is called the Data Memory. When a program runs a load instruction (like lw), the processor reads a number from the Data Memory. When it runs a store instruction (like sw), the processor writes a number into the Data Memory. In this lab you will design this Data Memory yourself, using Verilog. This lab is open-ended. This means you are given a goal and the rules, but you decide how to build it. You must explain why and how you choose your way.

What is Data Memory?

Think of the Data Memory as a big cupboard with many numbered boxes. Each box holds one 32-bit number. The box number is called the address. If you know the address, you can put a number in the box (write) or look at the number inside the box (read).

The following block diagram has four inputs and one output.

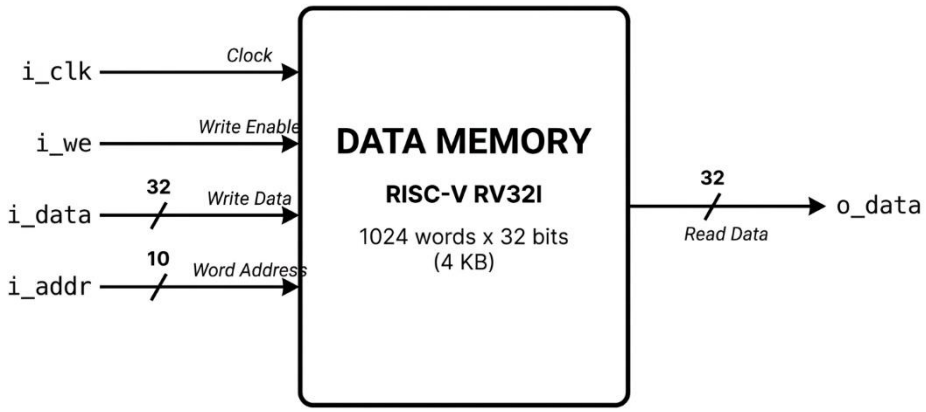


Figure 1: Block Diagram

Port	I/O	Width	What it does
i_clk	Input	1 bit	The clock. Writing happens on the rising edge of this clock.
i_we	Input	1 bit	Write Enable. 1 = write the data. 0 = do not write (only read).
i_data	Input	32 bits	The number that you want to store in memory.

i_addr	Input	10 bits	The address of the box (word) you want to use.
o_data	Output	32 bits	The number that is stored at the address i_addr.

Table 1: Data Memory I/O ports

The address has 10 bits. So we can make 2^{10} = 1024 different addresses (0 to 1023). Each address keeps a 32-bit word (4 bytes). So:

Total size = 1024 words x 4 bytes = 4096 bytes = 4 KB

The following figure 2 shows what is inside. There is a memory array, an address select part and a write control part (decides when to write).

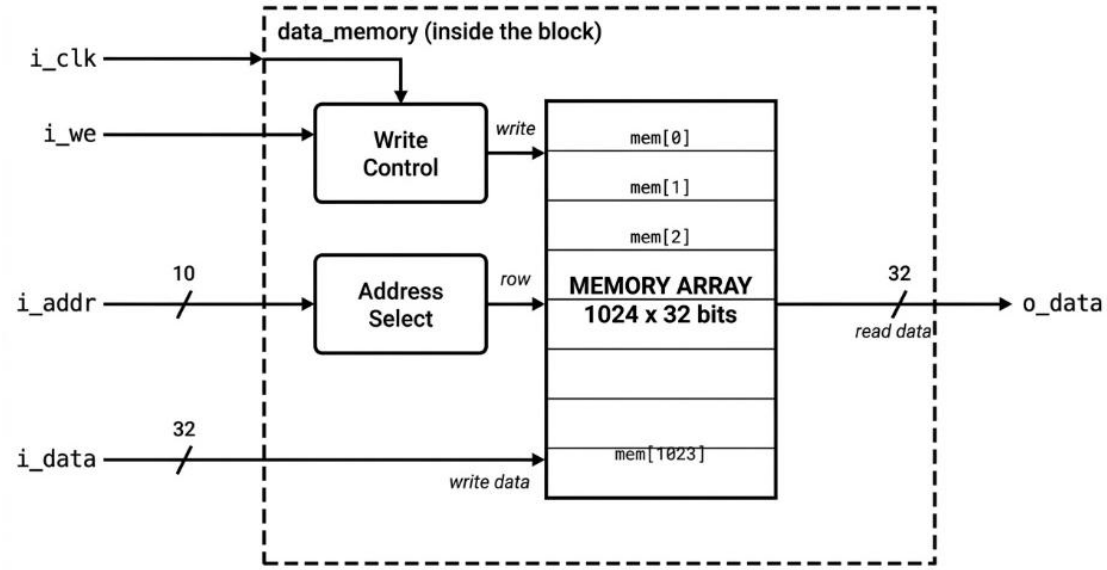


Figure 2: Data Memory Block Diagram

Figure 3 shows the Data Memory in a simple single-cycle RV32 processor. The ALU makes the address. The register file gives the data to store. The control unit gives the write enable. The output goes to the write-back mux and then to a register.

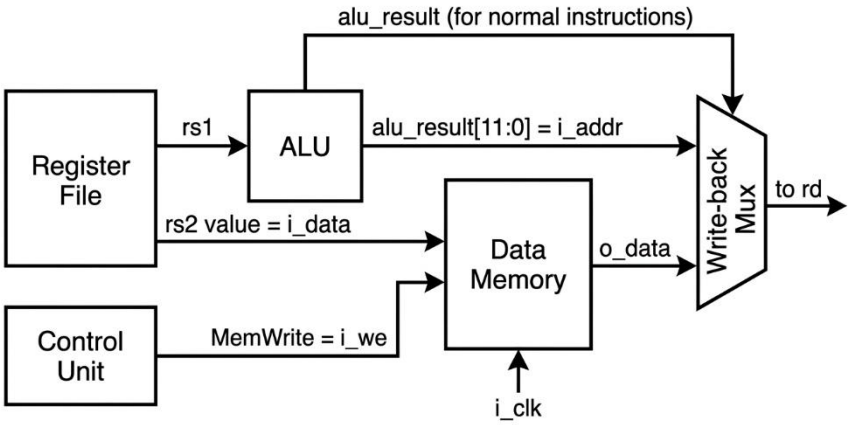


Figure 3: Data Memory of Single Cycle RV32 processor

Important: RISC-V uses byte addresses (each byte has its own address) and a word has 4 bytes. But our memory has word addresses. So we take bits [11:2] of the ALU result and send them to i_addr. We drop bits [1:0] because a word always starts at a multiple of 4. Example: byte address 0x00000014 (decimal 20) becomes word address 5.

Instruction	Meaning	What happens in Data Memory
sw rs2, imm(rs1)	Store Word	i_we = 1. i_data = rs2. Memory is updated at the clock edge.
lw rd, imm(rs1)	Load Word	i_we = 0. o_data goes to register rd.
sb / sh (extension)	Store Byte / Half-word	Only part of a word changes. Needs extra logic (see Task 7).
lb / lh / lbu / lhu (extension)	Load Byte / Half-word	Pick part of the word, then sign-extend or zero-extend (see Task 7).

Table 2: Load Store Instructions

Task

Design, code, test and explain a Data Memory module for a RISC-V processor. Use the ports shown in Figure 1. Your module must work correctly in a single-cycle processor.

- **Module name:** `data_memory`
- **Port names and widths:** exactly as in the table 1.
- **Size:** 1024 words of 32 bits.
- **Write:** happens only on the rising edge of i_clk and only when i_we = 1.
- **Read:** o_data must show the word stored at i_addr.
- **No** `x` values on o_data for any address after reset or start-up.
- **Code must be synthesizable** (no delays like #10 inside the design file).

Data Memory (.v) Sketch

```
//------------------------------------------------------------
// File   : data_memory.v
// Lab    : RISC-V Data Memory  (SKETCH - complete it!)
// Name   : _________________________   Roll No: ____________
//------------------------------------------------------------
module data_memory (
    input  wire        i_clk,    // clock
    input  wire        i_we,     // write enable (1 = write)
    input  wire [31:0] i_data,   // data to be written
    input  wire [9:0]  i_addr,   // word address (0 to 1023)
    output wire [31:0] o_data    // data read from memory
);

    // 1) MEMORY ARRAY: 1024 locations, each 32 bits wide
    reg [31:0] mem [0:1023];

    // 2) Note: simulation never shows 'x'

    // 3) WRITE PORT: happens only on the clock edge
    always @(posedge i_clk) begin
```

```
            if (/* TODO: which signal allows a write? */) begin
                mem[/* TODO: which address? */] <= /* TODO: which data? */;
            end
        end

        // 4) READ PORT: choose ONE style and write why in your report
        //    Style A - asynchronous read (data appears at once)
        assign o_data = /* TODO: read from the array */;

        //    Style B - synchronous read (data appears after a clock edge)
        //    reg [31:0] r_data;
        //    always @(posedge i_clk) r_data <= mem[i_addr];
        //    assign o_data = r_data;

    endmodule
```

Test Bench Sketch

```
//-------------------------------------------------------------
// File : tb_data_memory.v   (SKETCH testbench)
//-------------------------------------------------------------

module tb_data_memory();

    // Signals that drive the design


    // Connect the design (DUT = Design Under Test)


    // Clock: period = 10 ns
    initial clk = 1'b0;
    always #5 clk = ~ clk;


    // Task: write one word


    // Task: read one word and check it


    // Test sequence
    initial begin


        // TODO TEST 2: write with i_we = 0 must NOT change memory
        // TODO TEST 3: first location (0) and last location (1023)
        // TODO TEST 4: overwrite the same address with new data
        // TODO TEST 5: write many addresses, then read all back
        // TODO TEST 6: your own special test


endmodule
```

ASSESSMENT RUBRICS:

LLO	Statement	Assessment Method	Exemplary (86–100%)	Proficient (71–85%)	Developing (56–70%)	Beginning (0–55%)
2	Implement processor components and datapaths for specified functional requirements.	Practical demonstration, test results and lab submissions	Independently implements components or datapaths that meet all specified functional requirements across the required test cases.	Implements functioning components or datapaths that meet most requirements, with minor errors or occasional assistance.	Implements partially functioning components or datapaths; several requirements remain unmet and frequent assistance is needed.	Cannot produce functioning components or datapaths for the specified requirements despite substantial assistance.
3	Demonstrate responsible conduct during individual and group laboratory activities.	Instructor observation during lab and project activities	Consistently follows lab rules and safety instructions, is punctual and respectful, handles equipment responsibly, and fulfils an equitable share of group responsibilities.	Usually follows lab rules and safety instructions, behaves respectfully, handles equipment responsibly, and contributes to group work; occasional reminders are needed.	Needs repeated reminders about punctuality, rules, respectful behaviour or equipment care; contribution to group responsibilities is inconsistent.	Frequently disregards rules or instructions, disrupts others or handles equipment irresponsibly, and contributes little to group responsibilities despite reminders.

Lab Learning Outcomes	Total Marks	Obtained Marks
LLO 2	**10**	
LLO 3	**10**	

The End ☺